

[개발자 포트폴리오 웹 서비스]

요구사항 분석서

- 양지우 -

깃허브 링크: <https://github.com/qkwltkwkd1/GitHub-Sync.git>

문서번호 : [양지우]요구사항분석서_260503_Doc-004

소 속 : 소프트웨어학과

학 번 : 2024125037

팀 원 : 양지우

교 수 : 최차봉 교수님

제/개정 이력

버전	날짜	작성자 성명	제/개정사항	비 고
v1.0.0	2026-03-16	양지우	프로젝트 정의서 및 초기 폴더 구조 생성	
v1.1.0	2026-03-29	양지우	프로젝트 관리 계획서 초안 작성 및 README 업데이트	
v1.2.0	2026-04-11	양지우	요구사항 반영 프로젝트 관리 계획서 최종본 작성	
v1.3.0	2026-04-25	양지우	요구사항 정의서 작성	
v1.4.0	2026-05-03	양지우	요구사항 분석서(UML 모델링) 작성	

목차

1. 서론
 - 1.1 분석 모델의 목적
 - 1.2 분석 모델링의 기본 원칙
 - 1.3 시스템의 범위 및 정의
2. 기능 모델링
 - 2.1 유스 케이스 다이어그램
 - 2.1.1 시스템 경계 및 액터 식별
 - 2.1.2 유스 케이스 간 관계 정의
 - 2.2 유스 케이스 설명서
 - 2.2.1 UC-01: GitHub 계정 연동
 - 2.2.2 UC-02: 활동 데이터 동기화
 - 2.2.3 UC-03: 기술 스택 통계 조회 및 시각화
3. 구조 모델링
 - 3.1 클래스 식별
 - 3.1.1 문장 분석법을 통한 클래스 추출
 - 3.2 클래스 명세
 - 3.3 클래스 다이어그램
 - 3.3.1 클래스 속성 및 연산 정의
 - 3.3.2 클래스 간 관계성 정의
4. 행위 모델링
 - 4.1 순차 다이어그램
 - 4.1.1 데이터 동기화 시나리오 메시지 흐름
 - 4.1.2 제어 로직 표현
 - 4.2 상태기계 다이어그램
5. 분석 산출물 점검
 - 5.1 기능 모델과 구조 모델의 일관성 확인
 - 5.2 기능 모델과 행위 모델의 일관성 확인
 - 5.3 구조 모델과 행위 모델의 일관성 확인

1. 서론

1.1 분석 모델의 목적

- 고객의 요구사항을 객체지향 관점에서 상세히 기술하여 시스템의 'What'을 정의함
- 소프트웨어 설계의 기초를 확립하고 구축 후 검증 가능한 기준을 제공함

1.2 분석 모델링의 기본 원칙

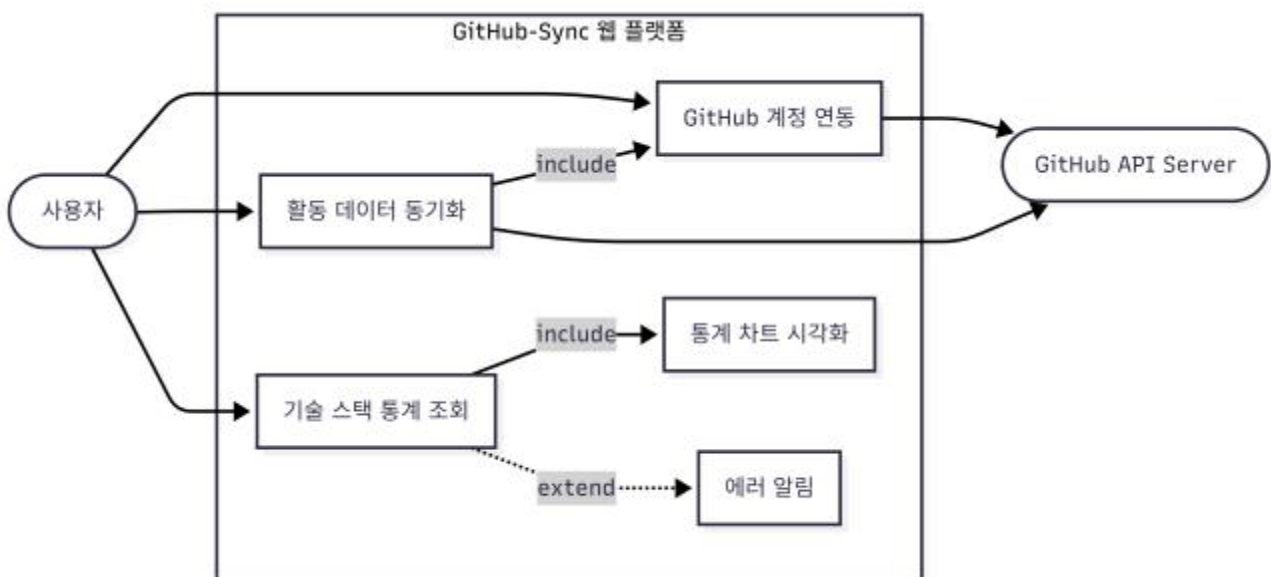
- 비기능적 요소나 인프라 환경보다는 비즈니스 도메인 내의 가시적인 요구사항에 집중함
- 높은 수준의 추상화를 유지하여 이해관계자 간의 원활한 의사소통을 지원함

1.3 시스템의 범위 및 정의

- 본 시스템은 GitHub API를 통해 사용자의 활동 데이터를 수집하고, MySQL DB를 활용하여 기술 스택을 분석·시각화하는 웹 기반 서비스를 범위로 함

2. 기능 모델링

2.1 유스 케이스 다이어그램



2.1.1 시스템 경계 및 액터 식별

- 시스템 경계: GitHub-Sync 웹 플랫폼.
- 주요 액터(Primary Actor): 사용자(User) - 데이터를 분석하고 조회하는 주체.
- 외부 액터(Secondary Actor): GitHub API Server - 연동되는 외부 소프트웨어 장치.

2.1.2 유스 케이스 간 관계 정의

- 포함(Include): 활동 데이터 동기화는 반드시 GitHub 계정 인증 과정을 거침.
- 확장(Extend): 통계 조회 시 데이터가 부족할 경우 에러 알림 기능이 선택적으로 실행됨.

2.2 유스 케이스 설명서

2.2.1 UC-01: GitHub 계정 연동

- 내용: OAuth 2.0 프로토콜을 사용하여 사용자의 GitHub 접근 권한을 안전하게 획득함.

2.2.2 UC-02: 활동 데이터 동기화

- 정상 시나리오:
 - 1) 사용자가 갱신 요청
 - 2) 시스템이 API 호출
 - 3) 수신 데이터를 MySQL 스키마로 가공
 - 4) DB 저장 및 완료 알림.

2.2.3 UC-03: 기술 스택 통계 조회 및 시각화

- 내용: DB에 저장된 언어별 비중 데이터를 Recharts 라이브러리를 통해 시각적 차트로 출력함.



3. 구조 모델링

3.1 클래스 식별

3.1.1 문장 분석법을 통한 클래스 추출

- 유스 케이스 시나리오의 일반 명사(사용자, 레포지토리, 커밋)를 클래스로 식별하고 동사(동기화, 저장)를 연산으로 매핑함.

3.2 클래스 명세

- Class: SyncController
- Responsibility: 데이터 가공, DB 트랜잭션 관리
- Collaborators: GitHubConnector, MySQL_DB

3.3 클래스 다이어그램

3.3.1 클래스 속성 및 연산 정의

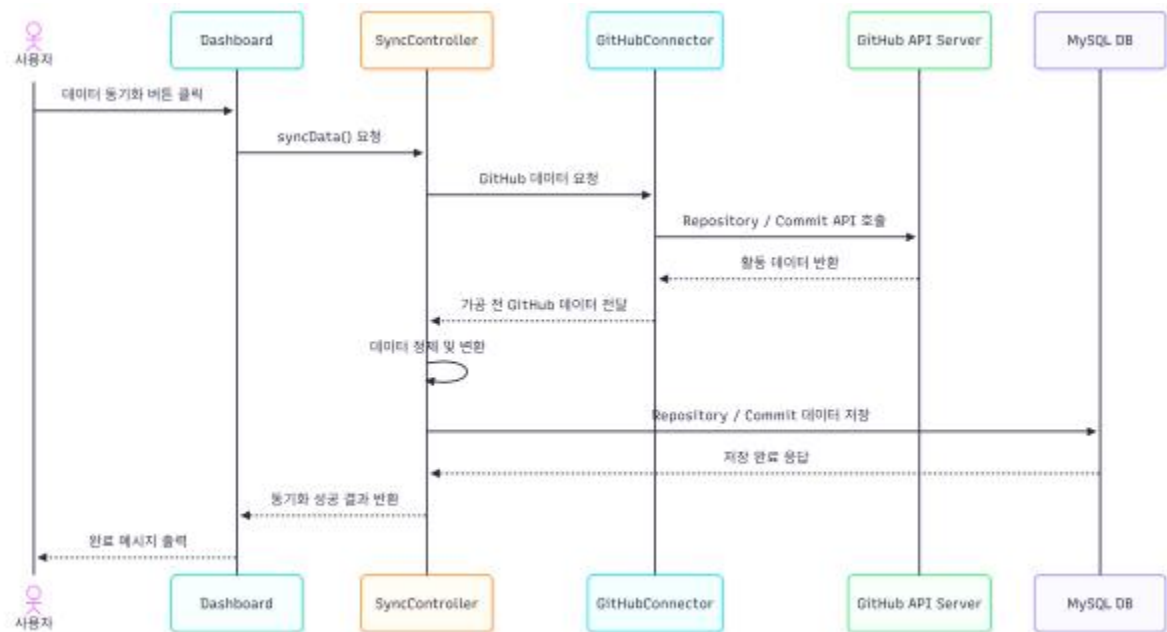
- User: +id: String, +syncData(): void
- Repository: -repoName: String, +calculateStats(): void.

3.3.2 클래스 간 관계성 정의

- 일반화(is-a): Person 클래스를 상속받는 User 구조
- 합성(has-a): Repository 객체가 소멸하면 내부 Commit 객체도 함께 소멸하는 관계

4. 행위 모델링

4.1 순차 다이어그램

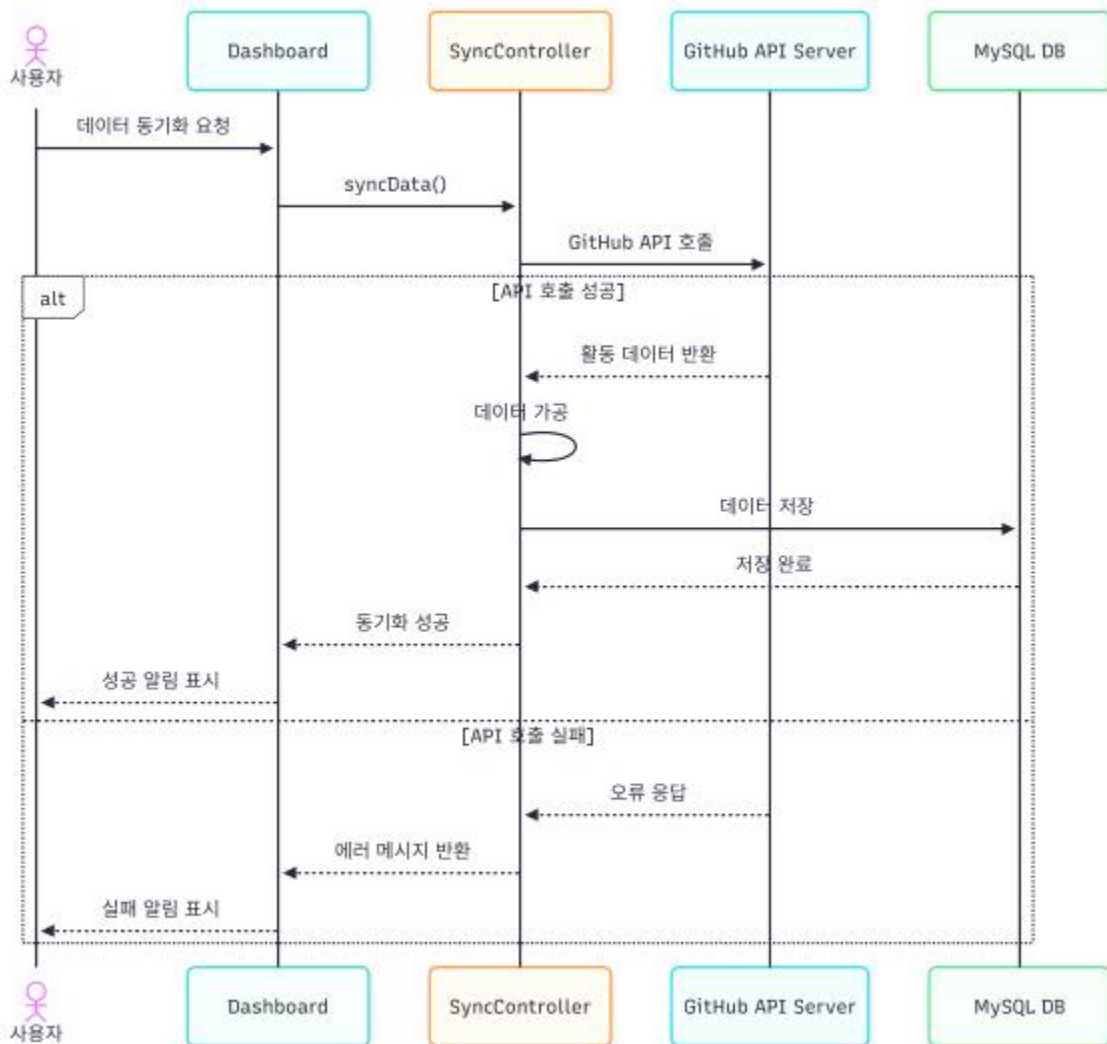


4.1.1 데이터 동기화 시나리오 메시지 흐름

- 사용자 → Dashboard → SyncController → API Connector → MySQL 순으로 메시지가 전달

4.1.2 제어 로직 표현

- Alt 심볼: API 통신 성공 시 Update_Success, 실패 시 Error_Handling 로직으로 분기 처리함.



4.2 상태기계 다이어그램

- 대상: User 객체.
- 상태: LoggedOut → Authenticating → Idle → Syncing

5. 분석 산출물 점검

5.1 기능 모델과 구조 모델의 일관성 확인

- 모든 유스 케이스 시나리오에 나타난 명사들이 클래스 다이어그램의 속성으로 정의되어 있음을 점검함.

5.2 기능 모델과 행위 모델의 일관성 확인

- 유스 케이스 액터가 순차 다이어그램의 메시지 교환 액터와 동일함을 확인함.

5.3 구조 모델과 행위 모델의 일관성 확인

- 순차 다이어그램에서 주고받는 모든 메시지명이 클래스 다이어그램의 연산(Method)으로 명세되어 있음을 검증함.